Accolite Digital L1 Interview Guide: Java Backend Developer

This comprehensive technical guide provides exhaustive, production-grade answers to core Java, Java 8, structural coding challenges, and Spring Boot conceptual questions frequently discussed during technical rounds at Accolite Digital.

Section 1: Core Java

---

1. Why is String immutable in Java?

Immutability ensures that once a String object is instantiated, its state cannot be modified. Java enforces this for several critical architectural reasons:

- **String Constant Pool (SCP) Optimization:** Memory efficiency is achieved through string sharing. Multiple reference variables can point to a single instance in the pool. If strings were mutable, altering a string via one reference would catastrophically corrupt the values observed by other components.
- **Security Constraints:** Strings are heavily relied upon to pass parameters such as file paths, database connections, URLs, and network sockets. If strings were mutable, an attacker could bypass validation checks by changing the string contents right after the check (Time-of-Check to Time-of-Use flaw).
- **Thread Safety:** Because their underlying state cannot be modified, String instances are inherently thread-safe. They can be shared concurrently across multiple threads without the performance penalties associated with synchronized code blocks.
- **HashCode Caching:** The hash code of a string is frequently computed and cached inside the object structure at creation time (lazy initialization). This makes strings extremely high-performing when used as keys in hash-based collections such as HashMap and HashSet, as the hash calculation is executed exactly once.

2. Internal Working of HashMap in Java 8

A HashMap operates on the principle of hashing, utilizing an internal array of nodes (buckets) to store structural key-value entries. In Java 8, a critical performance optimization called **Treeification** was introduced:

- **Data Structures:** Elements are tracked as instance nodes containing four properties: int hash, K key, V value, and Node<K,V> next. When a specific bucket experiences extreme hash collisions, the single-linked list transitions into a balanced Red-Black Tree composed of TreeNode elements.
- **Treeification Mechanics:** If the number of nodes within a single bucket exceeds the TREEIFY_THRESHOLD = 8 and the overall capacity of the table array is at least MIN_TREEIFY_CAPACITY = 64, the linked list transforms into a Red-Black tree. This reduces search time from O(n) down to O(log n). If elements drop below the UNTREEIFY_THRESHOLD = 6 during a resize operation, it converts back into a standard linked list.
- **Hash Distribution and Masking:** The index position is determined via bitwise operations: `int hash = (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);` `int index = (n - 1) & hash;` The bitwise XOR shifts the higher-order bits downward to mix variable data uniformly, eliminating structural bucket distribution clustering.

3. Difference between HashMap and ConcurrentHashMap

Feature	HashMap	ConcurrentHashMap (Java 8+)
Thread Safety	Not thread-safe; requires external manual synchronization when shared across threads.	Fully thread-safe; designed specifically for high concurrent data modifications.
Locking Strategy	None; completely un-synchronized.	Fine-grained bucket-level locking. Uses Compare-And-Swap (CAS) for empty buckets and locks only the bucket head via synchronized blocks on collisions.
Null Support	Permits one single null key and multiple null values.	Prohibits both null keys and null values (throws NullPointerException to prevent ambiguous states).

Feature	HashMap	ConcurrentHashMap (Java 8+)
Iterator Behavior	Fail-Fast; throws ConcurrentModificationException on structural mutations.	Fail-Safe (Weakly consistent); allows ongoing mutations without throwing exceptions.

4. Explain equals() and hashCode() Contract

The contract establishes rules for how objects must behave when used together in hash-based hashing frameworks:

1. If two objects are evaluated as equal via the equals(Object) logic, executing hashCode() on both instances **must** return the exact same integer value.
2. If two objects yield matching integers via hashCode(), they are **not** fundamentally required to be equal via equals(). This behavior is termed a hash collision.

Consequence of Violating the Contract: If equals() is customized but hashCode() is ignored, a collection like HashMap will locate different storage bucket array positions during retrieval attempts, causing duplicated keys or retrieving null even if an identical key exists.

5. Can we override static methods?

No, static methods cannot be overridden. Method overriding is dependent on dynamic execution binding at runtime based on the actual object instance type. Static methods, conversely, are coupled strictly to the enclosing class type rather than an explicit instance. They are bound at compile-time using compile-time polymorphism (static binding).
If a child class declares an identical static signature matching a parent method, this action is termed **Method Hiding**. The method version that executes is determined strictly by the reference type declared on the caller variable, not the underlying instance.

6. Difference between Abstract Class and Interface

Feature	Abstract Class	Interface (Java 8+)
Inheritance Model	Classes are bound by single inheritance (can extend exactly one parent class).	Classes can implement multiple interfaces simultaneously.
State & Variables	Can declare instance variables, constants, and fields with varying scope visibility.	Can only declare implicit variables that are forced as public static final constants.
Constructors	Permits constructor definitions to initialize abstract or concrete base state.	Cannot contain constructors.
Method Definitions	Can have abstract, concrete, protected, or private instance routines.	Allows abstract methods, default methods, static methods, and private helper variations.

7. What is the purpose of volatile?

The volatile keyword solves memory visibility issues in multithreaded systems:

- **Memory Visibility:** It instructs the JVM that reads and writes targeting the field must bypass local CPU caches and interact directly with central main memory, guaranteeing that modifications made by one thread are immediately visible to all other threads.
- **Instruction Reordering Inhibition:** It places memory fences around reads and writes of the volatile reference, preventing compilers or underlying hardware execution units from optimizing instruction ordering around the variable (establishing a clear "happens-before" contract).
- **Non-Atomicity Caveat:** It does **not** make compound transformations atomic. For example, count++ requires an explicit lock or an atomic wrapper class like AtomicInteger to execute safely under concurrent access.

8. Explain Synchronization with an example

Synchronization prevents race conditions by enforcing mutual exclusion so that only one thread can access a critical block of code at a time:

```
public class SynchronizedCounter {
    private int count = 0;

    // Enforces lock acquisition using the intrinsic 'this' monitor
    public synchronized void increment() {
        count++;
    }

    public int getCount() {
        synchronized(this) {
            return count;
        }
    }
}
```

9. What causes ConcurrentModificationException?

This exception occurs when a collection structure is modified structurally (elements inserted, cleared, or appended) while an active iterator loop is traversing its contents, unless the mutation is performed directly via the iterator's own methods (e.g., iterator.remove()).
Internal Tracking: Collections maintain an internal tracker field named modCount. When an iterator instance is spawned, it takes a snapshot value named expectedModCount = modCount. During each call to next(), if any numerical variance is discovered between modCount and expectedModCount, a ConcurrentModificationException is immediately thrown.

10. Difference between Fail-Fast and Fail-Safe Iterators

Fail-Fast Iterators: Operate directly on the live, original data array. They immediately throw a ConcurrentModificationException if structural modifications take place during loop cycles. Examples include standard iterators for ArrayList, HashSet, and HashMap.
Fail-Safe Iterators: Operate either on an independent structural clone or a weakly-consistent view of the collection data. They allow ongoing concurrent modifications without raising runtime exceptions. Examples include iterators for CopyOnWriteArrayList and ConcurrentHashMap.

Section 2: Java 8 Stream API & Optionals

---

11. Difference between map() and flatMap()

- **map():** Implements a 1-to-1 mapping transformation. It processes a single stream value of type T and outputs a single processed value of type R. (e.g., converting a list of string elements into a stream of their respective lengths).
- **flatMap():** Implements a 1-to-many structural transformation combined with flattening. It maps an individual element into an intermediate stream, and then flattens all of these sub-streams into a single continuous sequence. This is useful for unwrapping nested collection matrices (e.g., converting Stream<List<String>> down to a continuous Stream<String>).

12. Find duplicate elements using Streams

```
import java.util.*;
import java.util.stream.Collectors;

public class StreamDuplicates {
    public static void main(String[] args) {
        List<Integer> items = Arrays.asList(10, 24, 10, 33, 24, 56, 78);

        Set<Integer> trackSet = new HashSet<>();
        Set<Integer> duplicates = items.stream()
            .filter(n -> !trackSet.add(n)) // HashSet.add yields false if element exists
            .collect(Collectors.toSet());

        System.out.println(duplicates); // Outputs: [10, 24]
    }
}
```

13. Find second highest salary using Streams

```
import java.util.*;

public class SalaryStream {
    public static void main(String[] args) {
        List<Employee> employees = getEmployeeList();
```

```
        Optional<Double> secondHighest = employees.stream()
            .map(Employee::getSalary)
            .distinct()
            .sorted(Comparator.reverseOrder())
            .skip(1)
            .findFirst();
    }
}
```

14. Group employees by department using Streams

```
import java.util.*;
import java.util.stream.Collectors;

public class GroupingExample {
    public static void main(String[] args) {
        List<Employee> employees = getEmployeeList();

        Map<String, List<Employee>> empByDept = employees.stream()
            .collect(Collectors.groupingBy(Employee::getDepartment));
    }
}
```

15. Difference between Optional.orElse() and orElseGet()

- **orElse(T other):** This method evaluates the argument expression eagerly. The fallback instantiation logic inside orElse() runs even if the wrapping Optional contains a valid non-empty object value, incurring unnecessary overhead.
- **orElseGet(Supplier<? extends T> supplementary):** This method implements lazy evaluation. The functional lambda expression is only evaluated if the target Optional container is explicitly empty, making it more efficient for expensive object allocations.

Section 3: Coding Round Algorithms & System Design

---

16. Reverse words in a String without using library methods

```
public class StringWordReversal {
    public static String reverseWords(String input) {
        if (input == null || input.length() == 0) return input;
        char[] chars = input.toCharArray();
        int len = chars.length;

        // Step 1: Reverse the entire continuous character sequence
        reverseRange(chars, 0, len - 1);

        // Step 2: Reverse individual word segments boundaries
        int wordStart = 0;
        for (int i = 0; i < len; i++) {
            if (chars[i] == ' ') {
                reverseRange(chars, wordStart, i - 1);
                wordStart = i + 1;
            } else if (i == len - 1) {
                reverseRange(chars, wordStart, i);
            }
        }
        return new String(chars);
    }

    private static void reverseRange(char[] arr, int left, int right) {
        while (left < right) {
            char temp = arr[left];
            arr[left] = arr[right];
            arr[right] = temp;
            left++; right--;
        }
    }
}
```

17. Find the first non-repeating character

```
public class FirstUniqueChar {
    public static char findFirstUnique(String input) {
        int[] freq = new int[256]; // ASCII optimization mapping array
        for (int i = 0; i < input.length(); i++) {
            freq[input.charAt(i)]++;
```

```
        }
        for (int i = 0; i < input.length(); i++) {
            if (freq[input.charAt(i)] == 1) {
                return input.charAt(i);
            }
        }
        return '\0';
    }
}
```

18. Find the longest substring without repeating characters

```
import java.util.HashSet;

public class LongestUniqueSubstring {
    public static int getLongestSubstringLength(String text) {
        int left = 0, right = 0, maxLength = 0;
        HashSet<Character> elements = new HashSet<>();

        while (right < text.length()) {
            if (!elements.contains(text.charAt(right))) {
                elements.add(text.charAt(right));
                maxLength = Math.max(maxLength, right - left + 1);
                right++;
            } else {
                elements.remove(text.charAt(left));
                left++;
            }
        }
        return maxLength;
    }
}
```

19. Check if two strings are anagrams

```
public class AnagramVerification {
    public static boolean checkAnagram(String val1, String val2) {
        if (val1.length() != val2.length()) return false;
        int[] indexCounts = new int[256];
```

```
        for (int i = 0; i < val1.length(); i++) {
            indexCounts[val1.charAt(i)]++;
            indexCounts[val2.charAt(i)]--;
        }
        for (int val : indexCounts) {
            if (val != 0) return false;
        }
        return true;
    }
}
```

20. LRU Cache Design

An LRU (Least Recently Used) cache requires O(1) operations for both get and put actions. This is achieved by combining a **HashMap** with a custom **Doubly Linked List**:

```
import java.util.HashMap;

public class LRUCache {
    private class Node {
        int key, value;
        Node prev, next;
        Node(int k, int v) { this.key = k; this.value = v; }
    }

    private final int capacity;
    private final HashMap<Integer, Node> cacheIndex;
    private final Node head, tail;

    public LRUCache(int cap) {
        this.capacity = cap;
        this.cacheIndex = new HashMap<>();
        head = new Node(0, 0);
        tail = new Node(0, 0);
        head.next = tail;
        tail.prev = head;
    }

    public int get(int key) {
        if (!cacheIndex.containsKey(key)) return -1;
        Node entry = cacheIndex.get(key);
        detachNode(entry);
        moveToHead(entry);
        return entry.value;
```

```
    }

    public void put(int key, int value) {
        if (cacheIndex.containsKey(key)) {
            Node entry = cacheIndex.get(key);
            entry.value = value;
            detachNode(entry);
            moveToHead(entry);
        } else {
            if (cacheIndex.size() == capacity) {
                cacheIndex.remove(tail.prev.key);
                detachNode(tail.prev);
            }
            Node incoming = new Node(key, value);
            cacheIndex.put(key, incoming);
            moveToHead(incoming);
        }
    }

    private void detachNode(Node node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    private void moveToHead(Node node) {
        node.next = head.next;
        node.next.prev = node;
        head.next = node;
        node.prev = head;
    }
}
```

21. Design Parking Lot (Low-Level Design Discussion)

When discussing a low-level object-oriented parking lot design, define structured domain boundaries clearly using core classes and design principles:

- **Core Entities & Classes:**
    - ParkingLot: A Singleton coordinator tracking floor topologies and status properties.
    - ParkingFloor: Contains distinct configurations of designated slots (e.g., Compact, Large, Motorbike, Handicap).
    - ParkingSpot: Tracks allocation status, unique identifiers, and supported vehicle categories.
    - Vehicle: An abstract base class extended by specific variations like Car, Truck, or Motorbike.

    - Ticket: Captures entry timestamps, designated lane allocations, and structural pricing data.
- **Applicable Design Patterns:**
    - **Singleton Pattern:** Enforces a single global orchestration instance for the primary ParkingLot class.
    - **Factory Pattern:** Generates instance objects for varying subclass profiles of Vehicle or ParkingSpot types.
    - **Strategy Pattern:** Implements decoupled billing and fee-calculation logic variations (e.g., hourly flat rates versus duration-tiered billing models).

22. Producer-Consumer Problem Implementation

```
import java.util.LinkedList;

public class SharedQueueBuffer {
    private final LinkedList<Integer> bufferList = new LinkedList<>();
    private final int limitSize = 5;

    public void produceElement(int item) throws InterruptedException {
        synchronized (this) {
            while (bufferList.size() == limitSize) {
                wait(); // Suspend process while buffer is full
            }
            bufferList.add(item);
            notifyAll(); // Signal consumer threads
        }
    }

    public int consumeElement() throws InterruptedException {
        synchronized (this) {
            while (bufferList.isEmpty()) {
                wait(); // Suspend process while buffer is empty
            }
            int processed = bufferList.removeFirst();
            notifyAll(); // Signal producer threads
            return processed;
        }
    }
}
```

23. Multi-threaded Counter Implementation

```
import java.util.concurrent.atomic.AtomicInteger;

public class SafeConcurrentCounter {
    // Uses low-level hardware CAS instructions for lock-free thread safety
    private final AtomicInteger sequenceNum = new AtomicInteger(0);

    public void safeIncrement() {
        sequenceNum.incrementAndGet();
    }

    public int fetchValue() {
        return sequenceNum.get();
    }
}
```

Section 4: Spring Boot Core Architecture

---

24. What happens internally when Spring Boot starts?

When executing the entry point method SpringApplication.run(Application.class, args), the framework orchestrates the following internal lifecycle events:

1. **Initializer Setup:** Identifies application profiles and classpath configurations using factories found in META-INF/spring.factories.
2. **Environment Preparation:** Creates and configures the ConfigurableEnvironment, binding active properties, profiles, and system environment variables.
3. **ApplicationContext Creation:** Instantiates the core application context container (e.g., AnnotationConfigServletWebServerApplicationContext for web runtimes).
4. **Component Scanning and Definition Loading:** The ConfigurationClassPostProcessor parses configuration files, scanning for stereotype markers beneath the @SpringBootApplication root package to construct metadata structures called BeanDefinitions.
5. **Embedded Container Initialization:** The context bootstraps embedded servlet engines like Tomcat or Jetty directly.
6. **Runner Invocation:** Finds and runs any registered CommandLineRunner or ApplicationRunner beans to complete startup.

25. How does Dependency Injection work?

Dependency Injection (DI) is a structural realization of Inversion of Control (IoC). Instead of components self-instantiating dependencies directly, object references are supplied to target beans by the container framework:

- The ApplicationContext maps out structural dependencies using BeanDefinitions.
- When target instances are initialized, Spring dynamically resolves dependencies by type or qualifying identifier name.
- The framework uses Java Reflection mechanisms to wire dependency instances directly into fields, constructors, or setter methods.

26. Why is Constructor Injection preferred?

Constructor injection is the industry standard recommendation over field or setter approach mechanisms due to several engineering benefits:

- **Immutability Support:** Dependencies can be declared as final, ensuring they cannot be mutated or changed post-initialization.
- **Compile-Time Dependency Safety:** Guarantees that an object cannot be instantiated in an incomplete or broken state; required dependencies must be provided at instantiation time, eliminating runtime NullPointerException flaws.
- **Testing Flexibility:** Simplifies unit testing by allowing developers to pass mock dependencies directly through the constructor without needing a heavy Spring container environment or complex reflection hacks.

27. Difference between @Component, @Service, and @Repository

All three are stereotype annotations, meaning any class marked with them becomes a Spring-managed bean within the container. However, they serve different layer-specific roles:

- **@Component:** The root generic stereotype annotation. It marks a class as a utility or bean candidate for automated component scans.
- **@Service:** A semantic specialization applied at the business logic or service layer. It adds no extra technical features over @Component, but communicates architectural intent.
- **@Repository:** A specialization used at the data access or DAO layer. It enables automatic database exception translation, converting low-level vendor-specific database exceptions (e.g., SQL exceptions) into Spring's uniform DataAccessException hierarchy.

28. How does @Transactional work internally?

Spring manages database transactions using **Aspect-Oriented Programming (AOP) Proxies**:

1. When a bean contains a method annotated with @Transactional, Spring does not inject the raw bean instance directly into callers. Instead, it wraps it within an auto-generated proxy object (using either CGLIB or JDK dynamic proxies).
2. When a caller executes the method, the call goes to the proxy first. The proxy opens a database transaction using the configured PlatformTransactionManager.
3. The proxy invokes the target method inside an internal try-catch context block.
4. If the method finishes successfully, the proxy automatically commits the transaction. If a runtime, unchecked exception (e.g., RuntimeException) is raised, it automatically rolls back the transaction.

Section 5: Database Architecture & Query Optimization

---

29. Find Nth Highest Salary

This can be solved either using SQL query definitions or Java 8 Streams directly depending on where the operation is offloaded:

SQL (Subquery approach):

```
SELECT Salary FROM Employee e1
WHERE N-1 = (SELECT COUNT(DISTINCT Salary) FROM Employee e2 WHERE e2.Salary > e1.Salary);
```

SQL (Modern Window Function approach):

```
SELECT Salary FROM (
  SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) as rnk
  FROM Employee
) WHERE rnk = N;
```

Java 8 Streams API:

```
Optional<Double> nthHighest = employees.stream()
  .map(Employee::getSalary)
  .distinct()
  .sorted(Comparator.reverseOrder())
  .skip(n - 1)
  .findFirst();
```

30. Difference between Clustered and Non-Clustered Index

Feature	Clustered Index	Non-Clustered Index
Physical Storage	Alters and dictates the physical storage order of the data rows on the disk.	Maintains a completely separate logical structure from the physical table data.
Leaf Node Content	Leaf nodes contain the actual concrete data rows themselves.	Leaf nodes contain row pointers or clustering keys pointing back to the data rows.
Quantity Limit	Strictly exactly one per table.	Can define multiple indexes per table based on access patterns.

31. Explain Database Normalization

Normalization is a systematic database design practice aimed at removing data redundancy and preventing data modification anomalies (Insertion, Update, and Deletion anomalies):

- **First Normal Form (1NF):** Data values within cells must be completely atomic. Repeating groups or multi-valued column rows are strictly prohibited.
- **Second Normal Form (2NF):** Must be in 1NF. Additionally, all non-key attributes must be fully functionally dependent on the entire composite primary key, completely eliminating partial dependency configurations.
- **Third Normal Form (3NF):** Must be in 2NF. Also, fields must have no transitive dependencies. A non-key attribute cannot establish a functional dependency on another non-key attribute.
- **Boyce-Codd Normal Form (BCNF):** A stronger version of 3NF. For any non-trivial functional dependency $X \rightarrow Y$, the determinant X must be a candidate super key.

32. How would you optimize a slow query?

Query optimization follows a structured diagnostic lifecycle:

1. **Analyze Execution Plans:** Run EXPLAIN ANALYZE on the target query statement to determine bottlenecks like Full Table Scans vs Index Range Scans.
2. **Evaluate Indexing Strategies:** Add appropriate B-Tree or composite indexes for columns utilized inside WHERE clauses, JOIN conditions, or ORDER BY operators.
3. **Refactor Query Semantics:** Eliminate wildcards (replace SELECT * with specific column listings), avoid bad operators like NOT IN or leading wildcards (LIKE '%xyz') which break index usage, and split large operations into manageable chunks.
4. **Connection Pooling & DB Properties:** Verify sizing configurations for tools like HikariCP and confirm that data fetch limits (such as JPA Hibernate batch fetching sizes) are enabled to mitigate N+1 fetch issues.
5. **Architectural Fallbacks:** Introduce distributed read replicas to split write traffic from read queries, set up caching using Redis for static hot data, or implement database partitioning/sharding for very large datasets.

Section 6: Distributed Systems, Kafka & Scaling

---

33. If Kafka is down, what will happen?

The systemic impact of a Apache Kafka broker failure depends on how producers and consumers are configured:

- **Impact on Producers:** In-memory message send buffers will begin filling up rapidly. If the cluster outage lasts longer than the time set in max.block.ms, producer threads will throw exceptions. If configurations use synchronous blocking writes, incoming API gateway calls will hang, potentially causing upstream timeouts. To prevent data loss, fallback patterns must be implemented to write messages to a local failover data bucket or relational database.
- **Impact on Consumers:** Consumers will fail to poll new record batches, but they can safely finish processing whatever records they already fetched in memory. Once Kafka comes back online, consumers will reconnect and resume reading messages from their last securely committed offset position.

34. How do you avoid duplicate message processing?

This is solved by designing an **Idempotent Consumer** pattern to guarantee that processing a message multiple times yields the same system state as processing it exactly once:

1. **Unique Correlation Identifiers:** Enforce that every event message contains a unique business identifier (such as a Transaction UUID).

2. **Deduplication Registry Check:** Before executing any business logic, the consumer attempts to write this unique ID into a distributed cache with an explicit TTL (like Redis) or a relational database table with a unique constraint.
3. **Atomic Commit or Skip:** If the database insert throws a unique constraint violation or Redis indicates the key already exists, the consumer immediately acknowledges the message to Kafka and skips processing to prevent duplicates.

35. How do you scale a microservice handling high traffic?

Scaling high-throughput microservices requires changes across multiple infrastructure layers:

- **Application Layer Scaling:** Deploy multiple containerized stateless service instances across a Kubernetes cluster. Use a Horizontal Pod Autoscaler (HPA) to scale pods automatically based on CPU utilization or custom application metrics.
- **Asynchronous Decoupling:** Offload resource-heavy or non-blocking tasks (like notification dispatches or statement generation) to message queues like Kafka to keep API response times low.
- **Caching Layer:** Deploy a Redis cluster to serve frequently read, slow-changing database records, reducing the read load on your primary databases.
- **Database Layer Scaling:** Implement a read-write architecture with read replicas, configure high-performance connection pooling via HikariCP, and use database sharding to distribute large tables across multiple database nodes.

Section 7: Production Scenario & Experience

---

36. Explain one complex production issue you resolved

Problem Statement: During a high-traffic promotional event, an account service began failing with connection timeouts, throwing HikariPool-1 - Connection is not available, request timed out after 30000ms errors, quickly followed by java.lang.OutOfMemoryError: Java heap space crashes.
Root Cause Investigation: We pulled heap dumps using jcmd and analyzed them with Eclipse Memory Analyzer (MAT). The analysis revealed that a newly deployed reporting feature was streaming huge datasets inside a active Spring @Transactional block. Because the database stream remained open while processing business logic, it held onto the underlying database connection indefinitely. Under high concurrent traffic, this caused connection pool starvation, and the large data structures held in memory rapidly exhausted the JVM heap space.
Resolution Strategy: We resolved the issue by implementing a multi-step fix:

- Refactored the data retrieval logic to use explicit pagination with Spring Data Pageable, ensuring

data was processed in small, manageable chunks instead of a single massive block.

- Moved the data processing out of the long-running @Transactional context block to release database connections back to the pool as quickly as possible.
- Wrapped the data access code in a proper try-with-resources block to guarantee that resources were safely closed even if exceptions occurred.
- Turned on HikariCP's leak-detection-threshold configuration to automatically track and log unclosed connection leaks in the future.